

Container Orchestration Kubernetes

In MEICAN

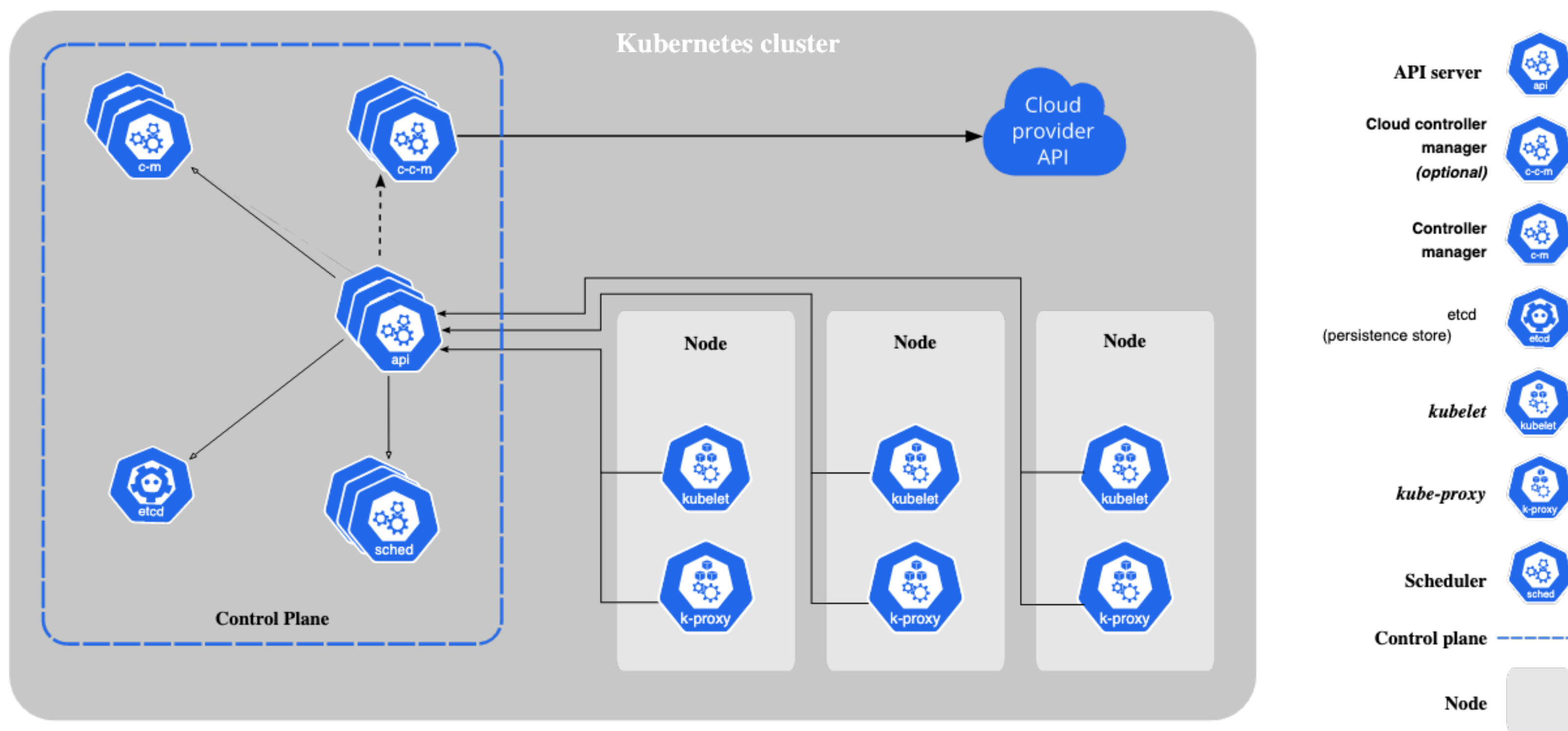
Table of Content

- What is Kubernetes ?
 - Overview
 - Control Plane
 - Workload
 - Service Discovery
- How we implement it in Meican ?
 - Architecture
 - Kong Ingress
 - Service Mesh - Istio
 - Pod
 - ArgoCD - Deployment Controller
- Rich Features
 - Logging / Monitoring / Tracing / IAM ...

What is Kubernetes ?

“κυβερνήτης”

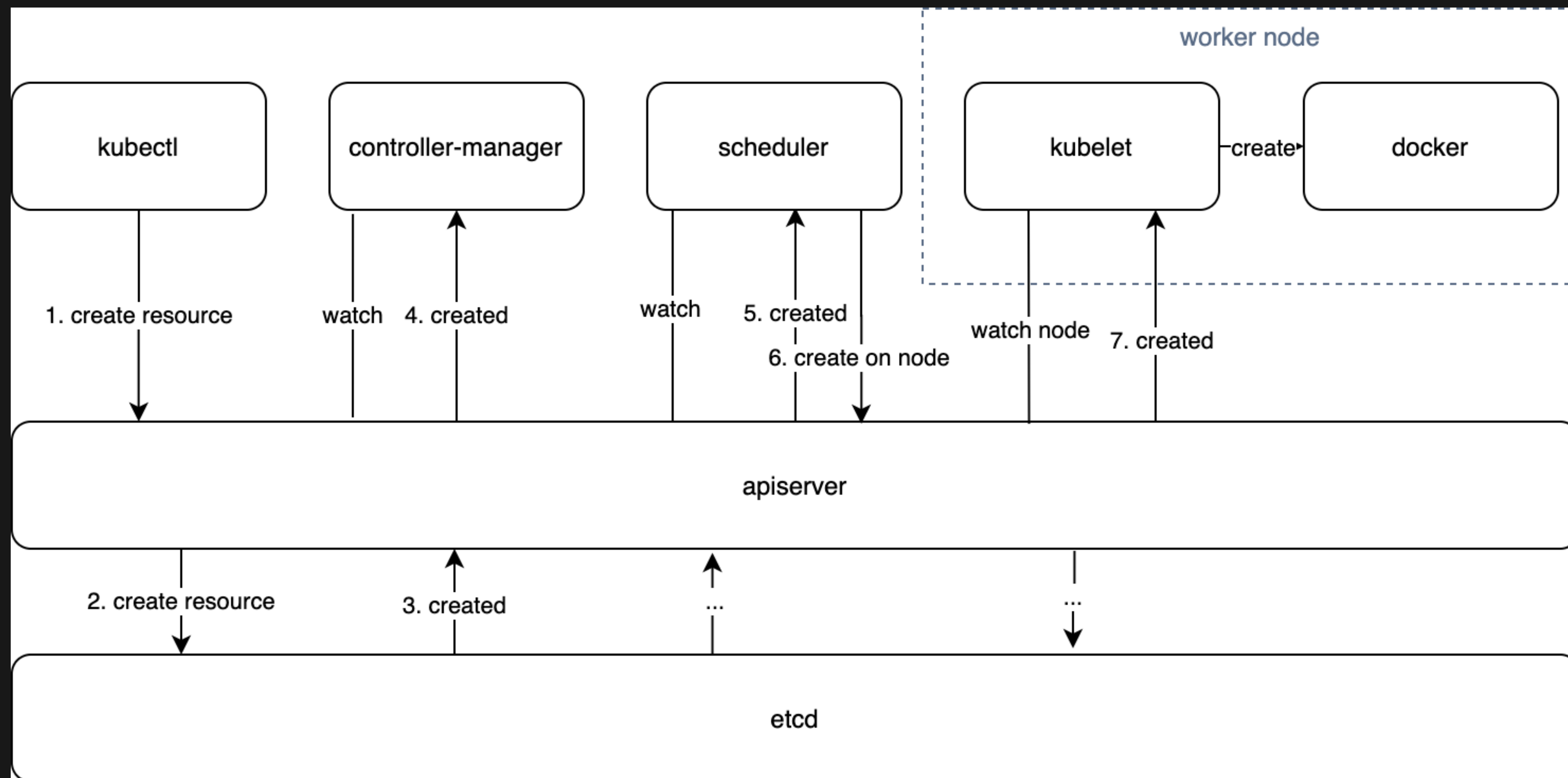
Greek Word



Kubernetes Overview

Control Plane

- kube-apiserver
 - Frontend
- etcd
 - Persistence
- kube-scheduler
 - Scheduling decision
- kube-controller-manager
 - Watch & control
- cloud-controller-manager
 - Interaction with cloud providers

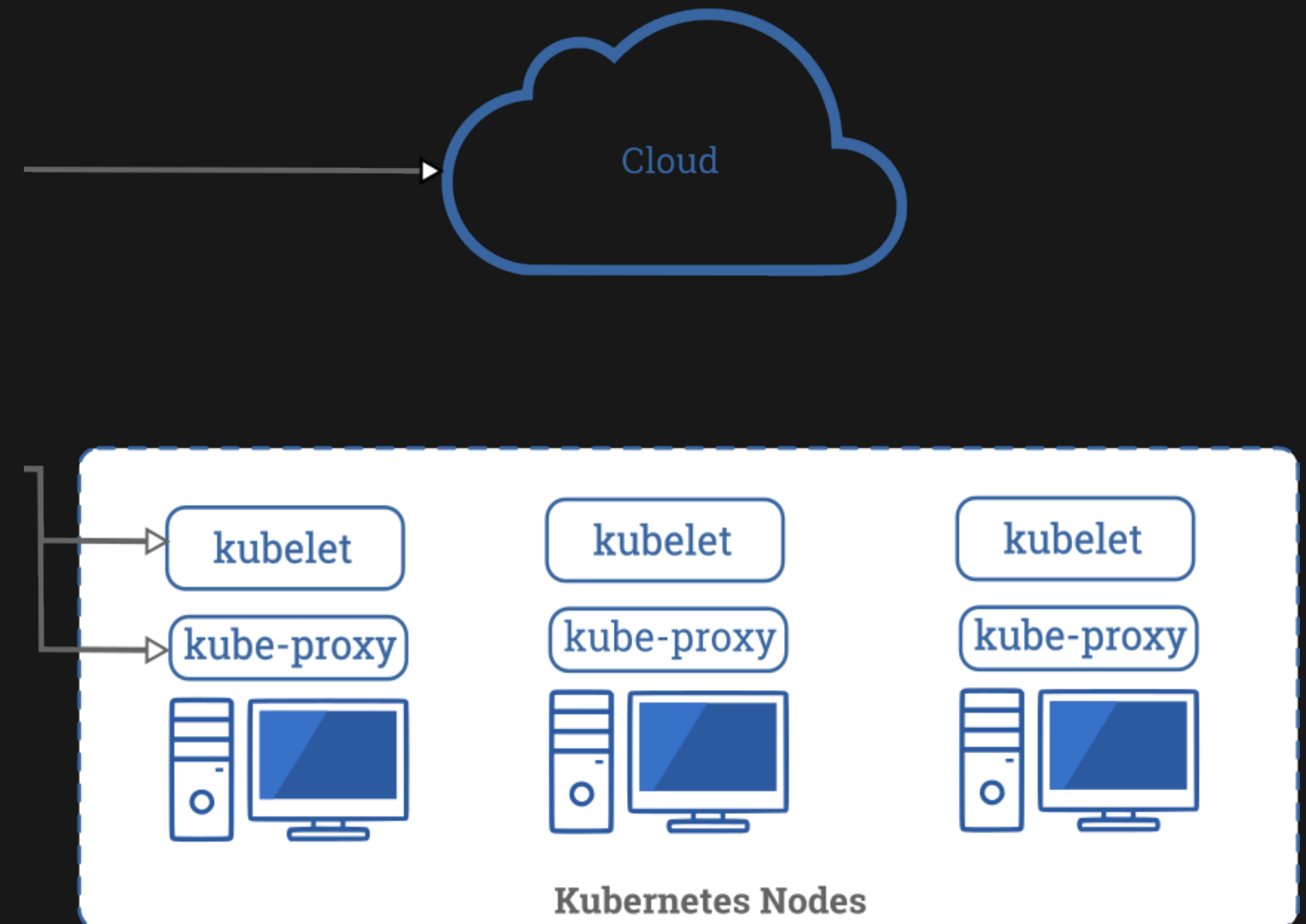


Kubernetes

Control Plane

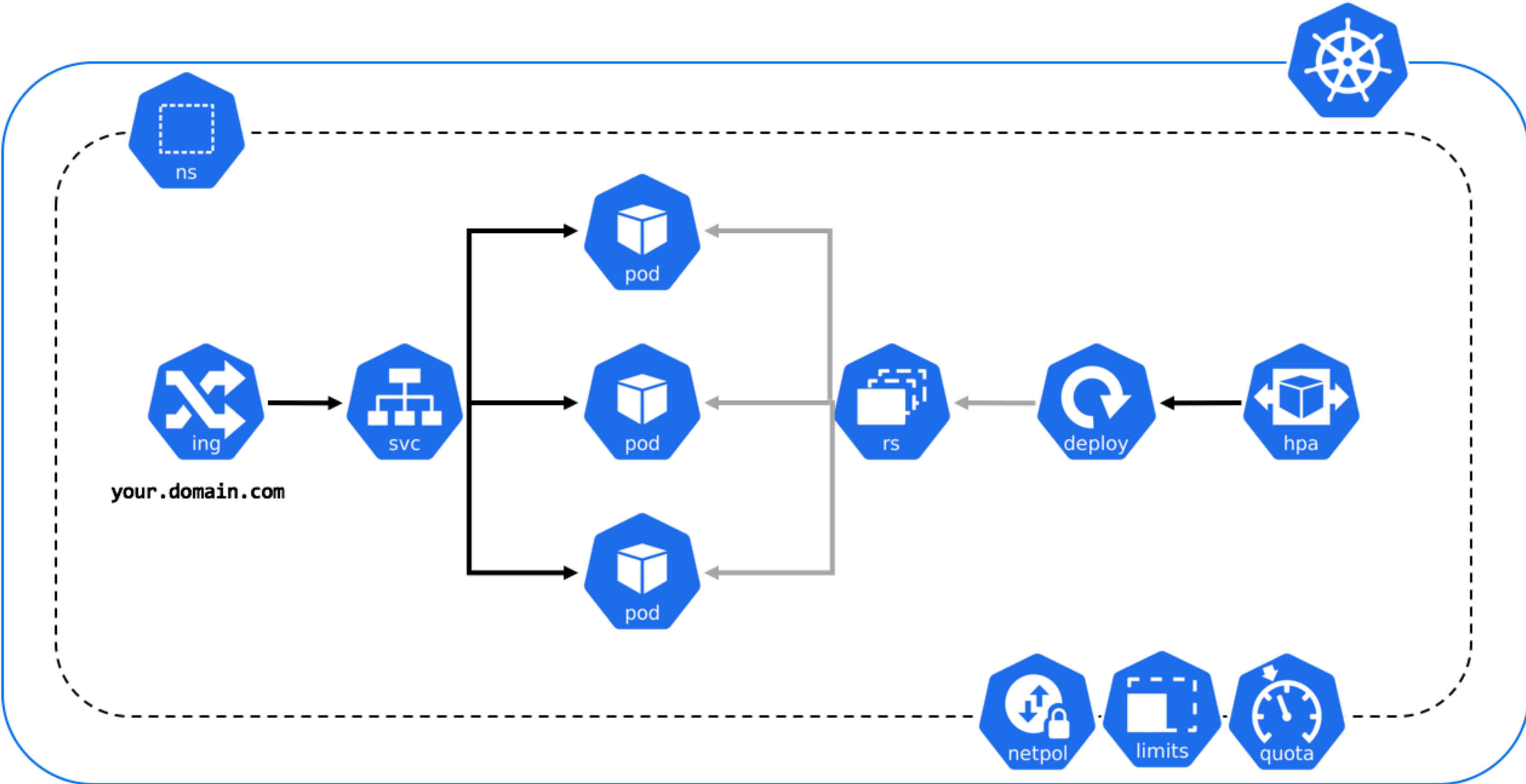
Control Plane

- kubelet
 - dispatcher
- kube-proxy
 - network
 - proxy or no-proxy
- CRI (Container Runtime Interface)



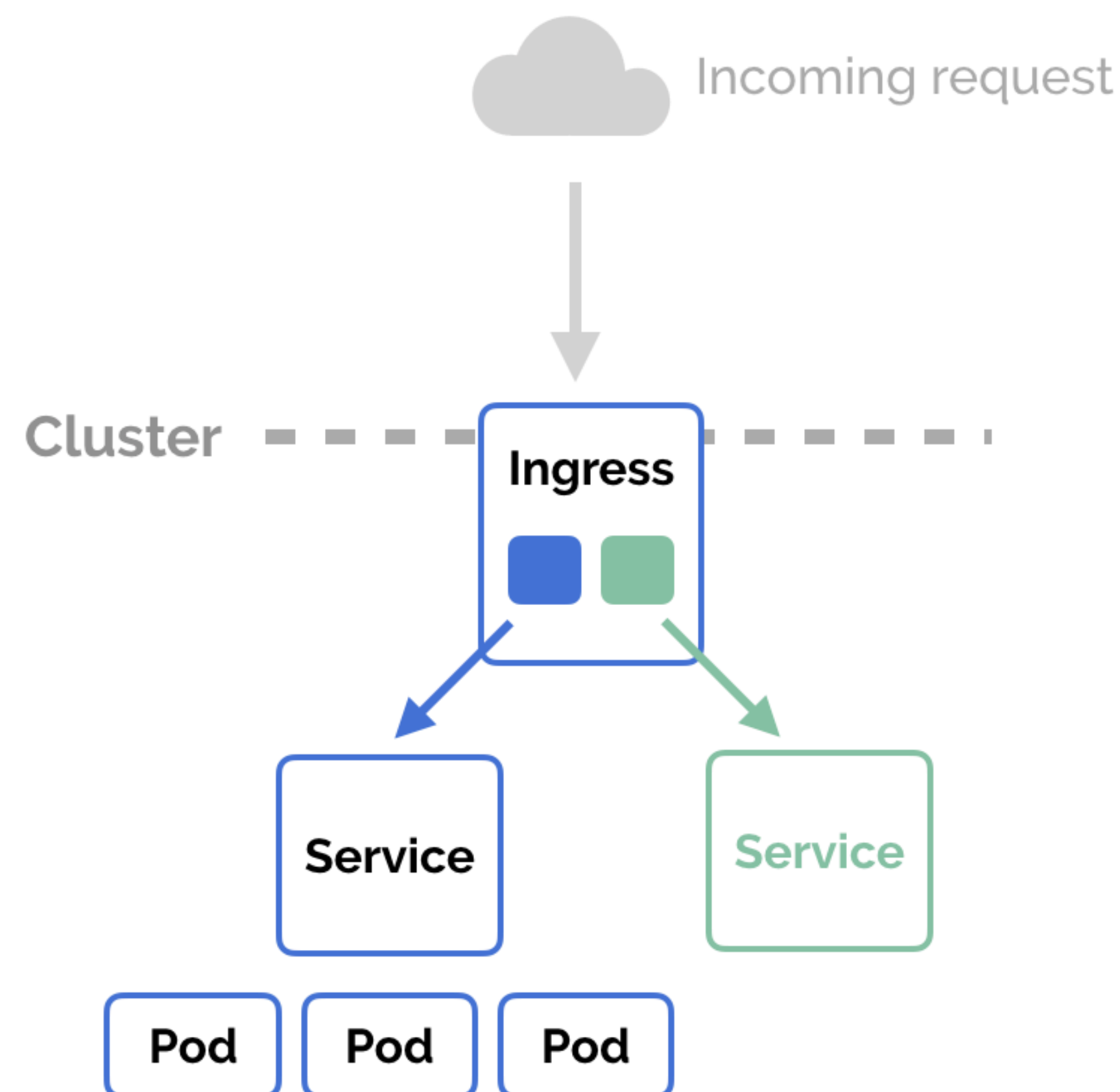
Workload

- Pod
- Deployment
 - Multiple pods
- ReplicaSet
- StatefulSet
 - Like Deployment
 - Bind Volume
- DaemonSet
 - Each Node
- Job
 - One time / periodic



Service Discovery

- Service
 - Inner cluster
 - “name.namespace.svc.cluster.local”
- Ingress
 - Outer cluster
 - Ingress Controller
- NodePort / LoadBalancer

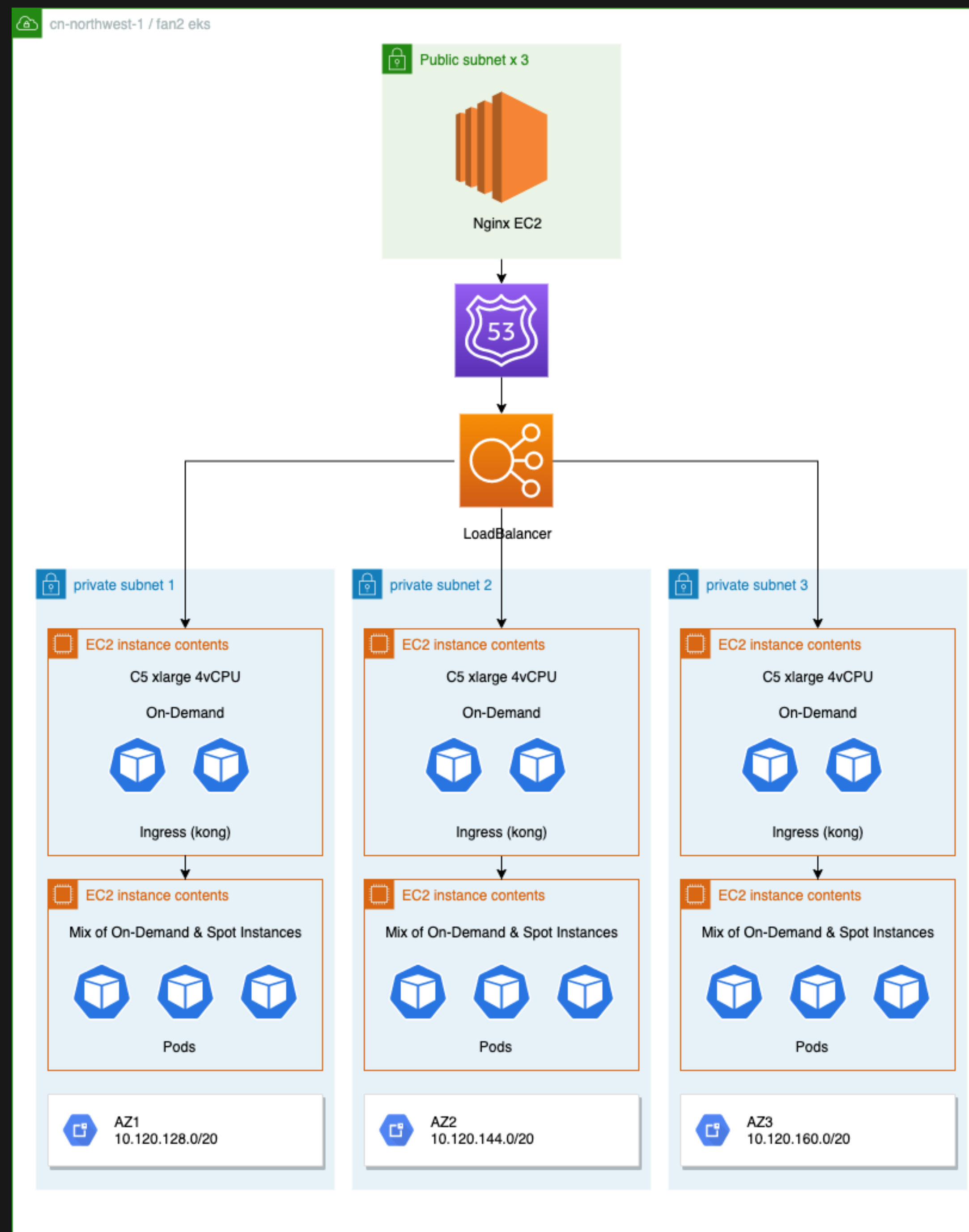


How we implement it ?

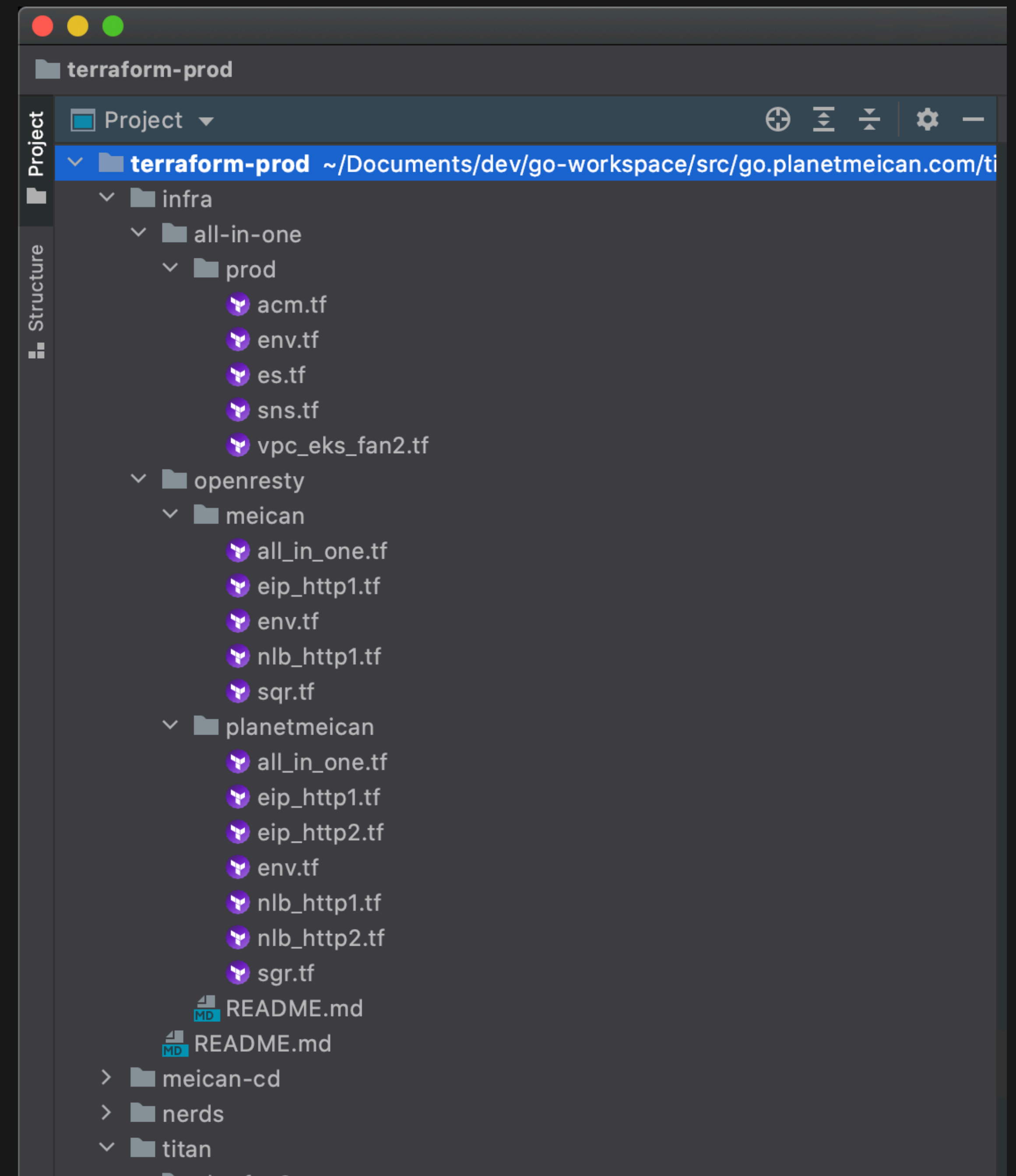
EKS Architecture

In Meican

Infrastructure AWS Resources

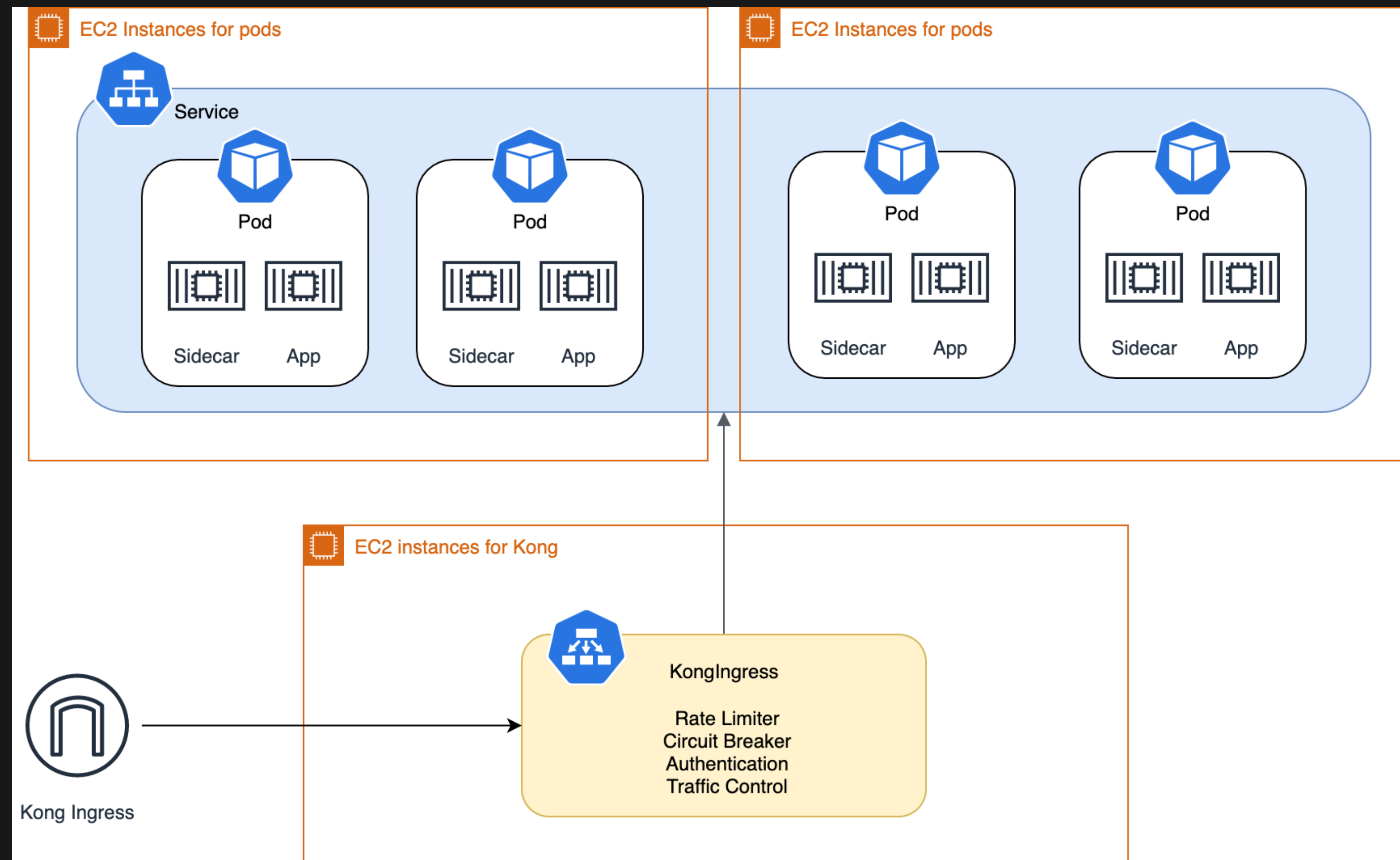


AWS Resources Powered by Terraform



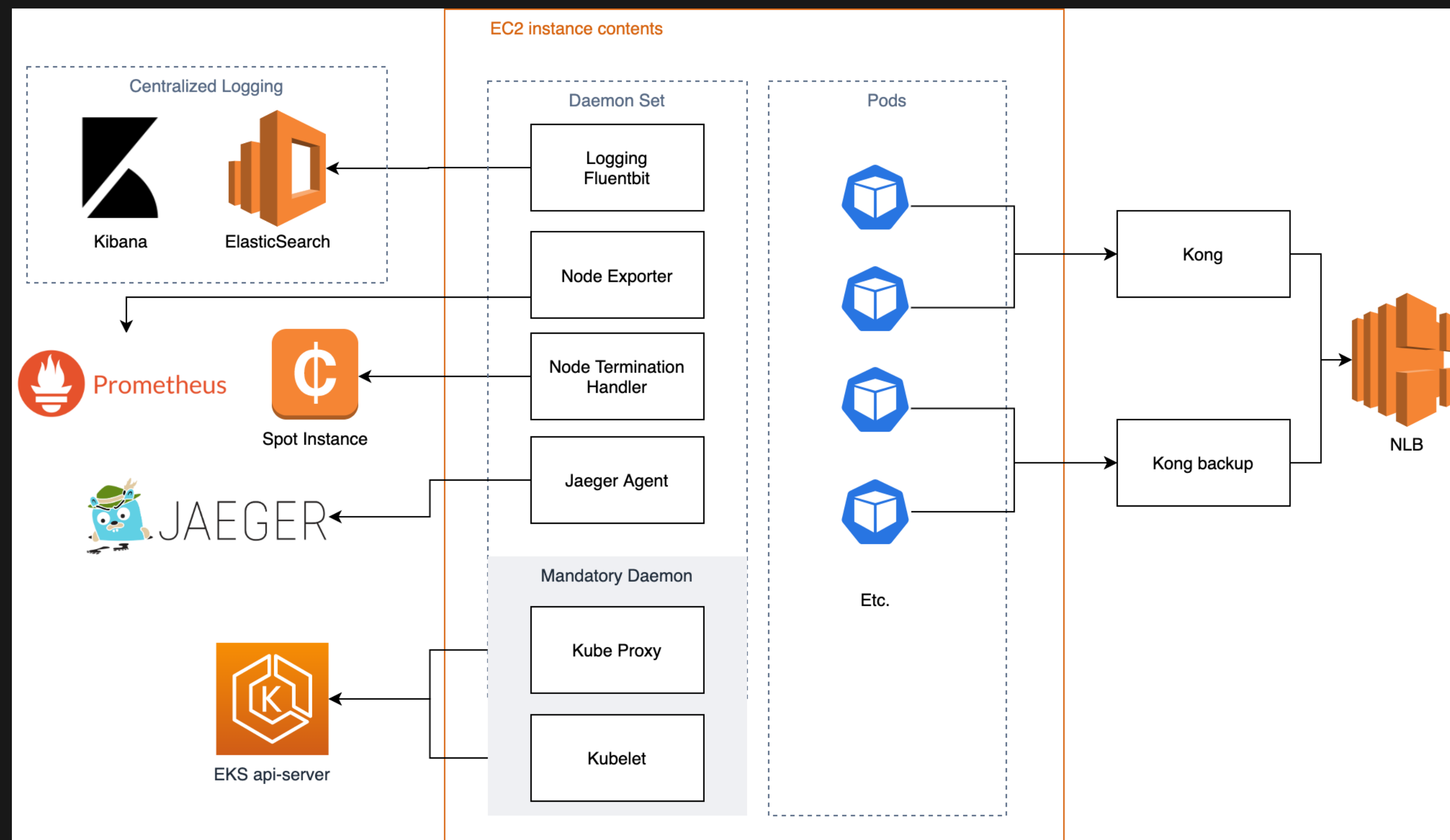
Available Clusters

- Sandbox:
 - Active Development
- Prod1:
 - eks-fan
 - Won't develop unless minor bug fixes
- Prod2:
 - eks-fan2
- Why fan2 ?
 - Portability with Terraform
 - Made for expedition



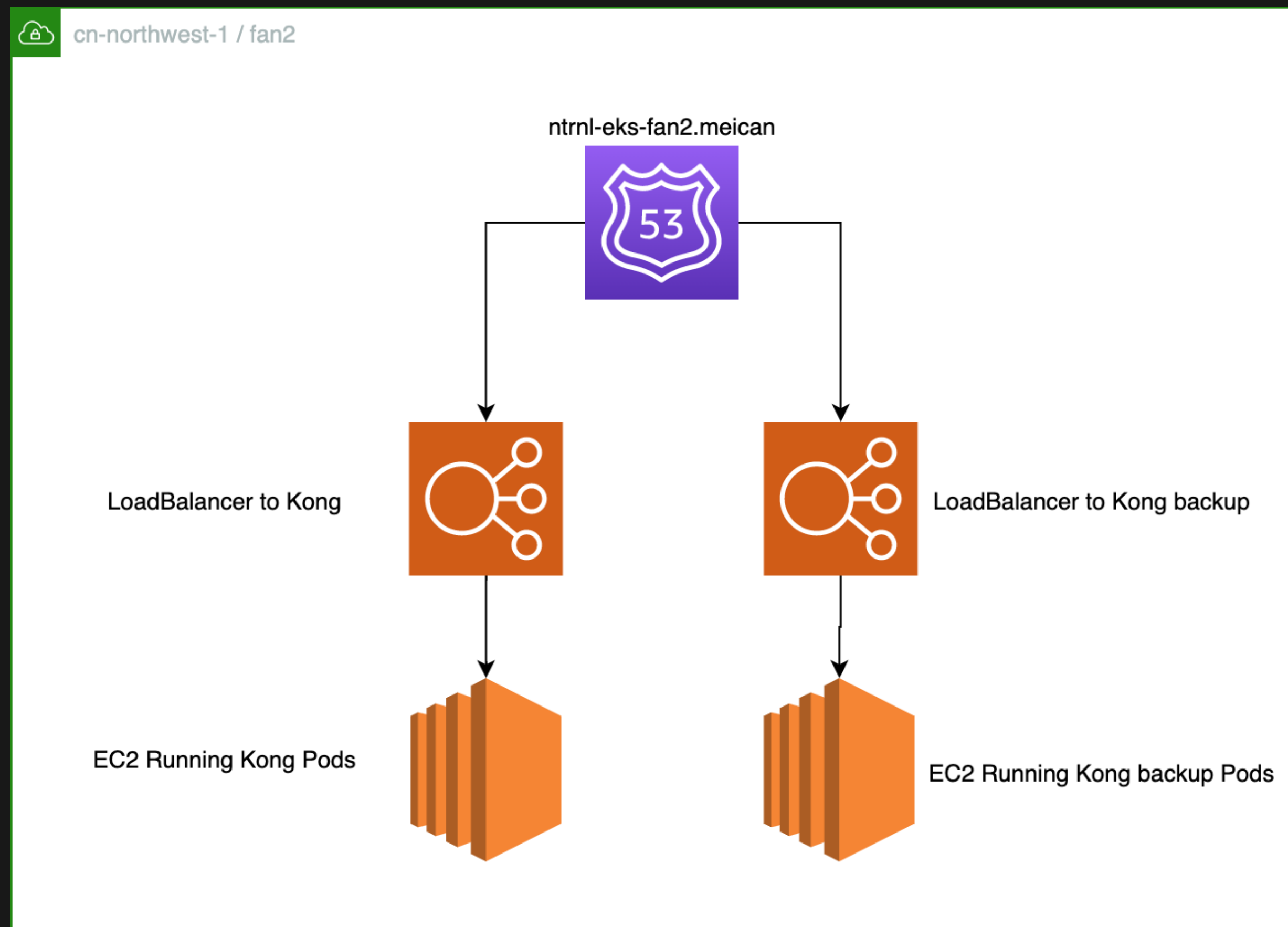
EC2 Worker Node

What's in it ?



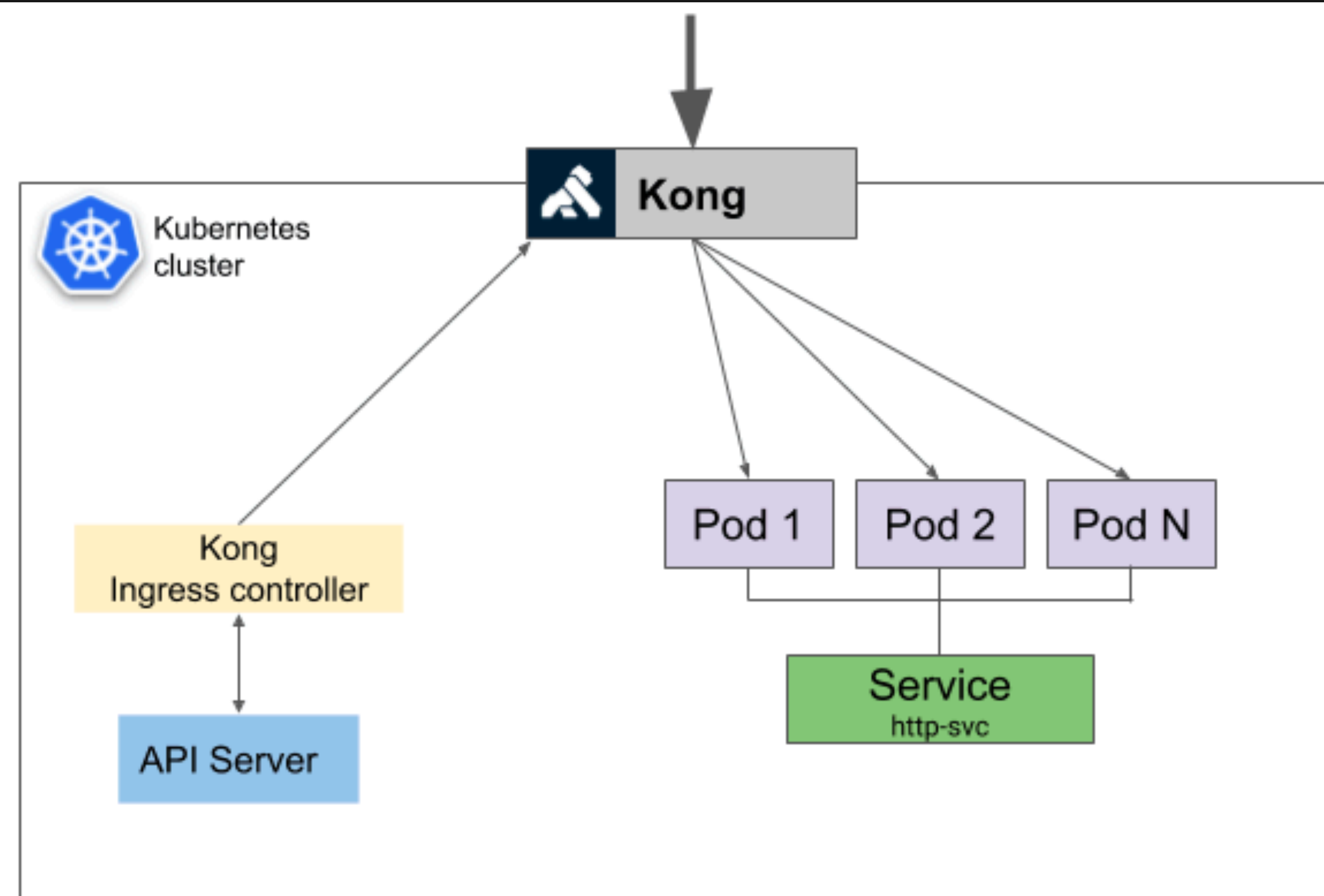
EC2 Worker Node

DaemonSet Components



Infrastructure

Ingress Controller



Kong Ingress

Lua + Nginx

Kong Ingress

- SOA Features
- Plugin Based
 - set-upstream
 - Rate Limiter
 - Circuit Breaker
 - Authenticating
 - Inbound / Outbound Control
 - Transformation
 - Logging
 - Monitoring

```

apiVersion: extensions/v1beta1 networking.k8s.io/
kind: Ingress v1
metadata:
  name: apple-kong
  namespace: app1
  annotations:
    kubernetes.io/ingress.class: "kong1"
    konghq.com/plugins: app1-apple-rate-limit
spec:
  rules:
    - host: apple.app1.ntrnl-eks-fan.meican
      http:
        paths:
          - path: /
            backend:
              serviceName: apple
              servicePort: 5678

```

⚠ Changed After K8S 1.18

```

apiVersion: configuration.konghq.com/v1
kind: KongPlugin
metadata:
  name: app1-apple-rate-limit
  namespace: app1
config:
  minute: 10
  policy: redis
  redis_host: kong-redis.7wwbbi.0001.cnw1.cache.amazonaws.com
plugin: rate-limiting

```

Kong Ingress

Example

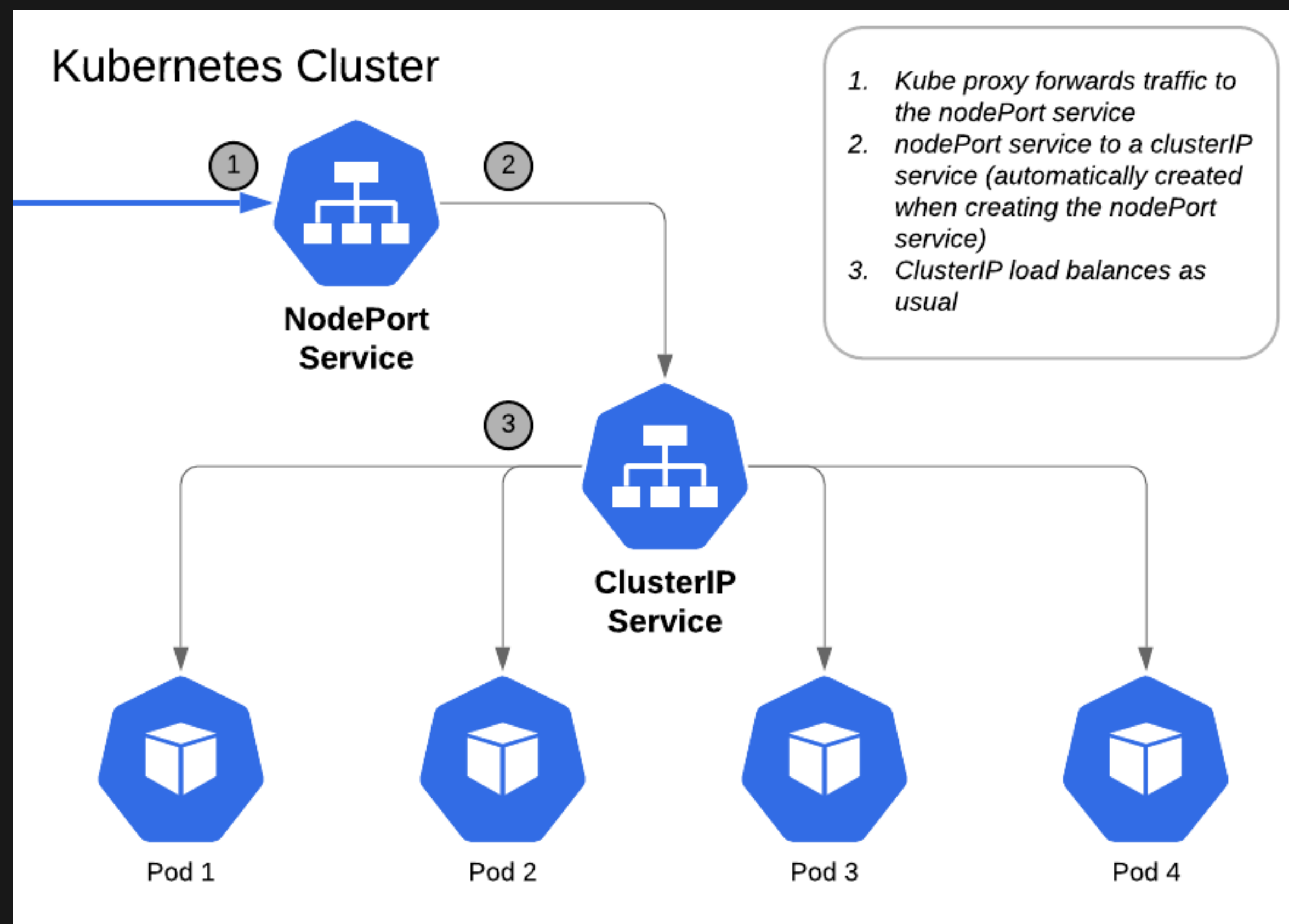
- Naming Convention
 - `{your-app}.app-{your-group}.ntrnl-eks-fan{clusterID}.meican`
- Example
 - `order20.app-mutants.ntrnl-eks-fan2.meican`

- Invocation via Kong Ingress
 - HTTP: port 80
 - gRPC: port 443 w/ InsecureSkipVerify

```

—— tlsOption = grpc.WithTransportCredentials(
——   credentials.NewTLS(&tls.Config{
——     InsecureSkipVerify: true,
——   }),
—— )
—— cc, _ := grpc.DialContext(
——   ctx,
——   target: "order20.app-mutants.ntnrl-eks-fan2.meican:443",
——   tlsOption,
—— )
—— client := clientmemberorderv1.NewServiceClient(cc)

```



Service

Cluster IP

Service Example

23:31:12

~

\$ kubectl -n testing get svc

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
apple	ClusterIP	172.20.140.126	<none>	5678/TCP	25d
fortune-teller	ClusterIP	172.20.98.192	<none>	50051/TCP	25d

service.yaml

```

1  kind: Service
2    apiVersion: v1
3    metadata:
4      name: order20
5      namespace: app-mutants
6    labels:
7      app: order20
8    annotations:
9      konghq.com/protocol: "grpc"
10   spec:
11     selector:
12       app: order20
13     ports:
14       - protocol: TCP
15         port: 80
16         targetPort: 8022
17         name: http
18       - protocol: TCP
19         port: 8023
20         targetPort: 8023
21         name: grpc
22       - protocol: TCP
23         port: 18888
24         targetPort: 18888
25         name: telemetry

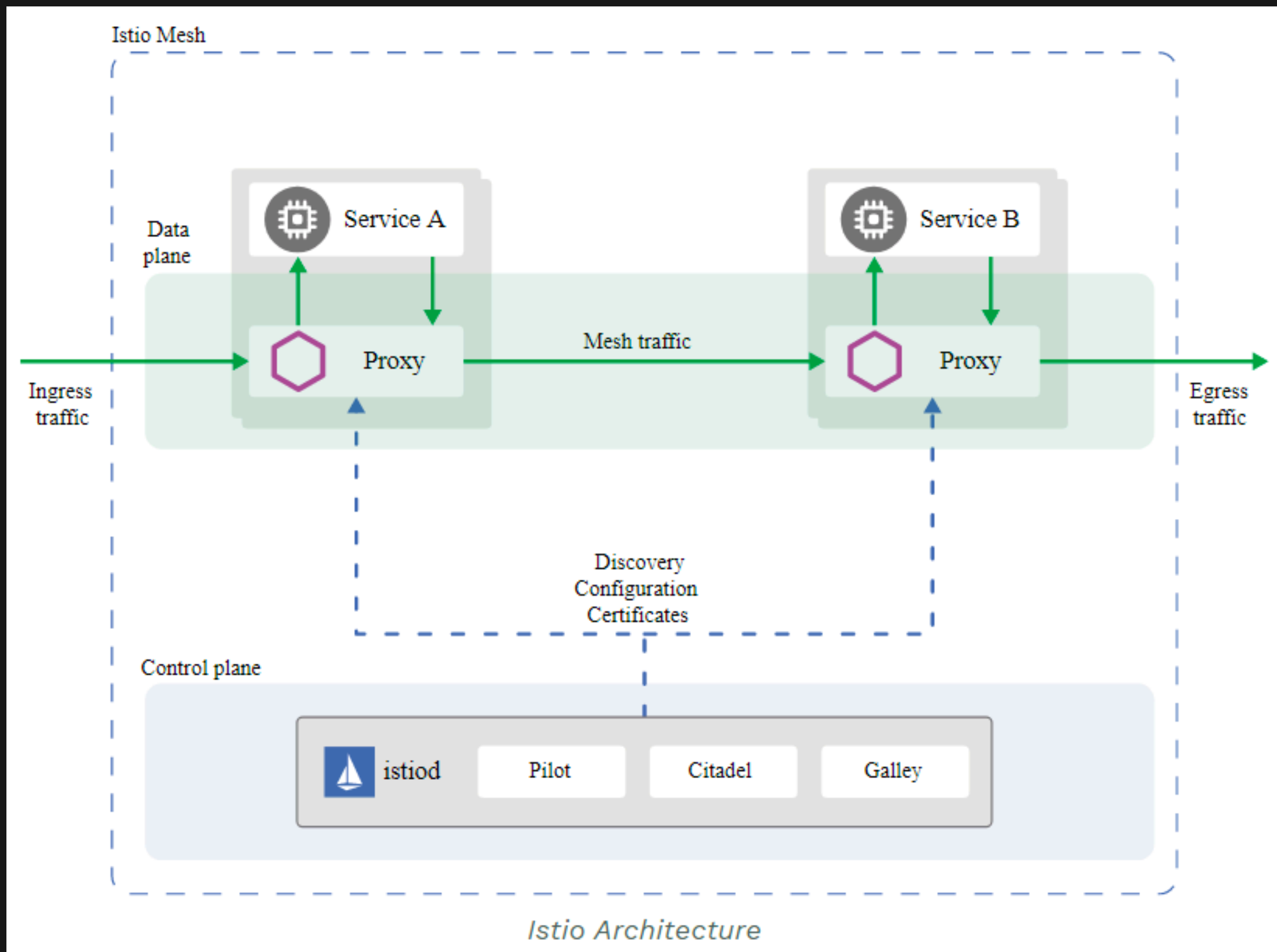
```

! match to pod

! match to protocol

! default telemetry port

- Invocation via Service
 - Port defined in target service
 - gRPC w/ DialOption WithInsecure

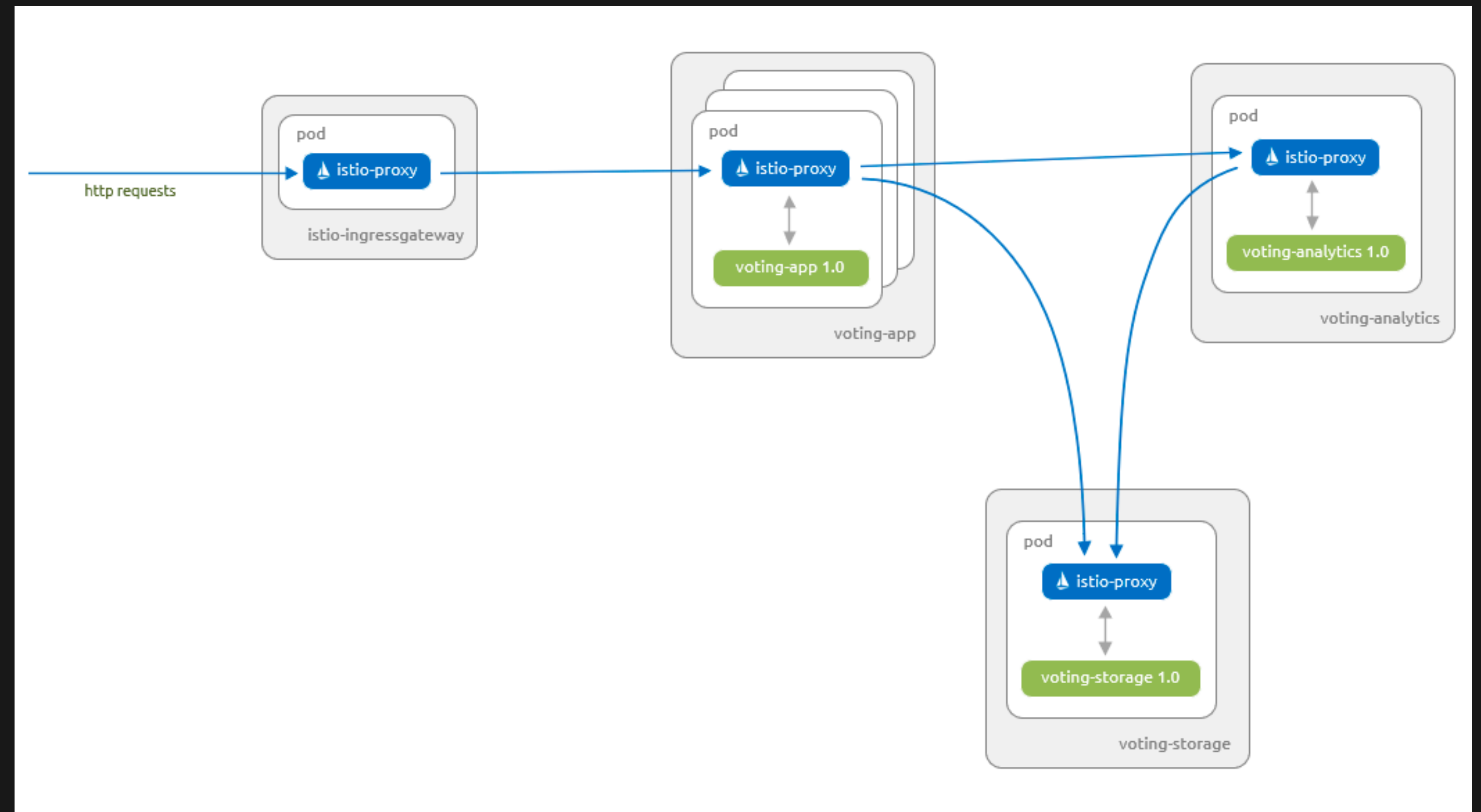


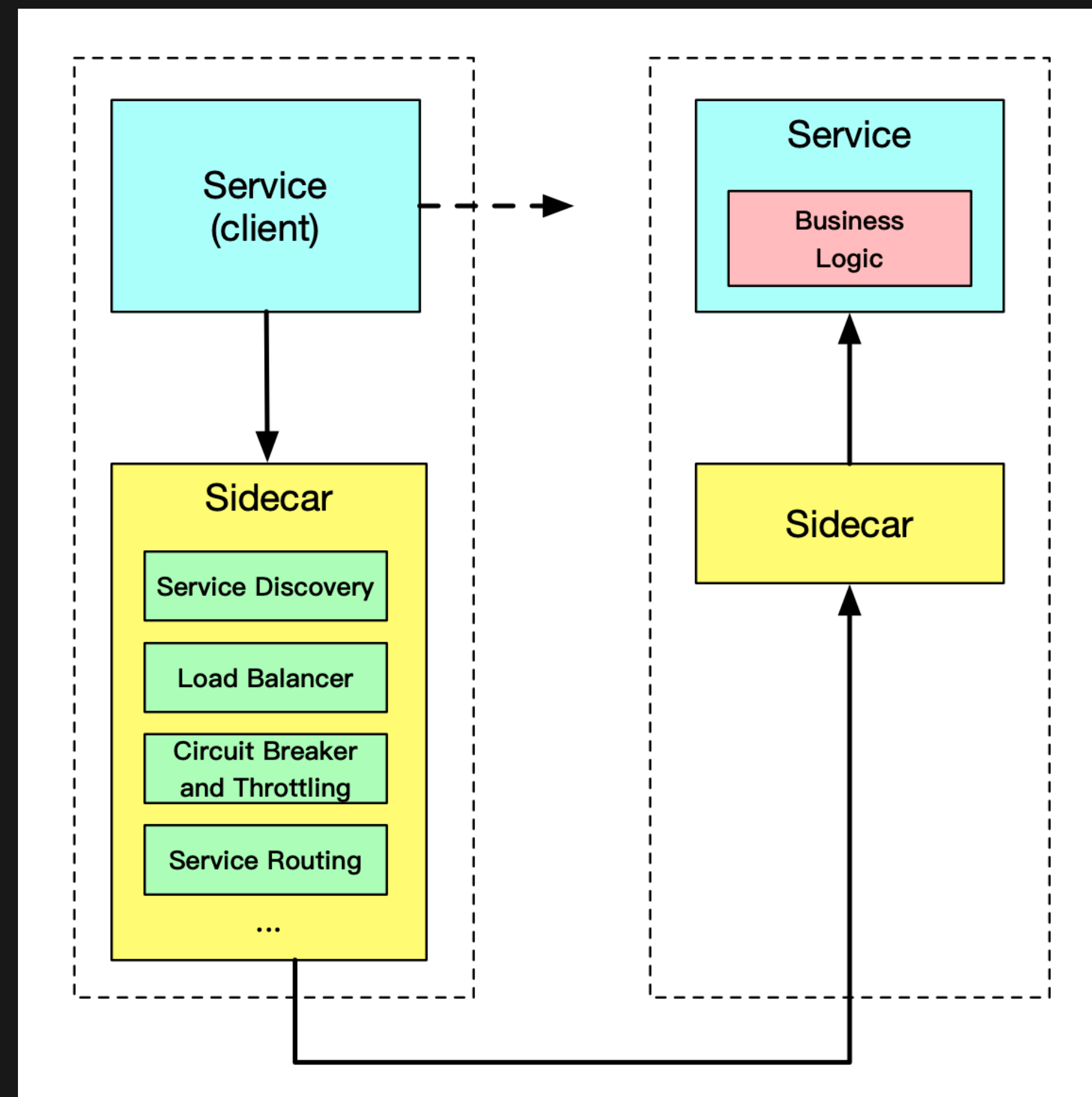
Istio

Service Mesh

Why VirtualService ?

- Routing Rules
- Load Balancing Options
- Timeout
- Retry
- Circuit Breaker
- Fault Injection
- etc.





```
istio-iptables -p 15001 -z 15006 -u 1337 -m REDIRECT -i '*' -x "" -b '*' -d 15090,15020
```

IPTable Hijacking

```

1  ► apiVersion: networking.istio.io/v1alpha3
2  kind: VirtualService
3  metadata:
4    name: order20-vs
5    namespace: app-mutants
6  spec:
7    hosts:
8      - order20.app-mutants.svc.cluster.local
9    http:
10     - name: primary
11       route:
12         - destination:
13             host: order20-stable
14             subset: v1
15             weight: 100
16         - destination:
17             host: order20-canary
18             subset: v1
19             weight: 0

```

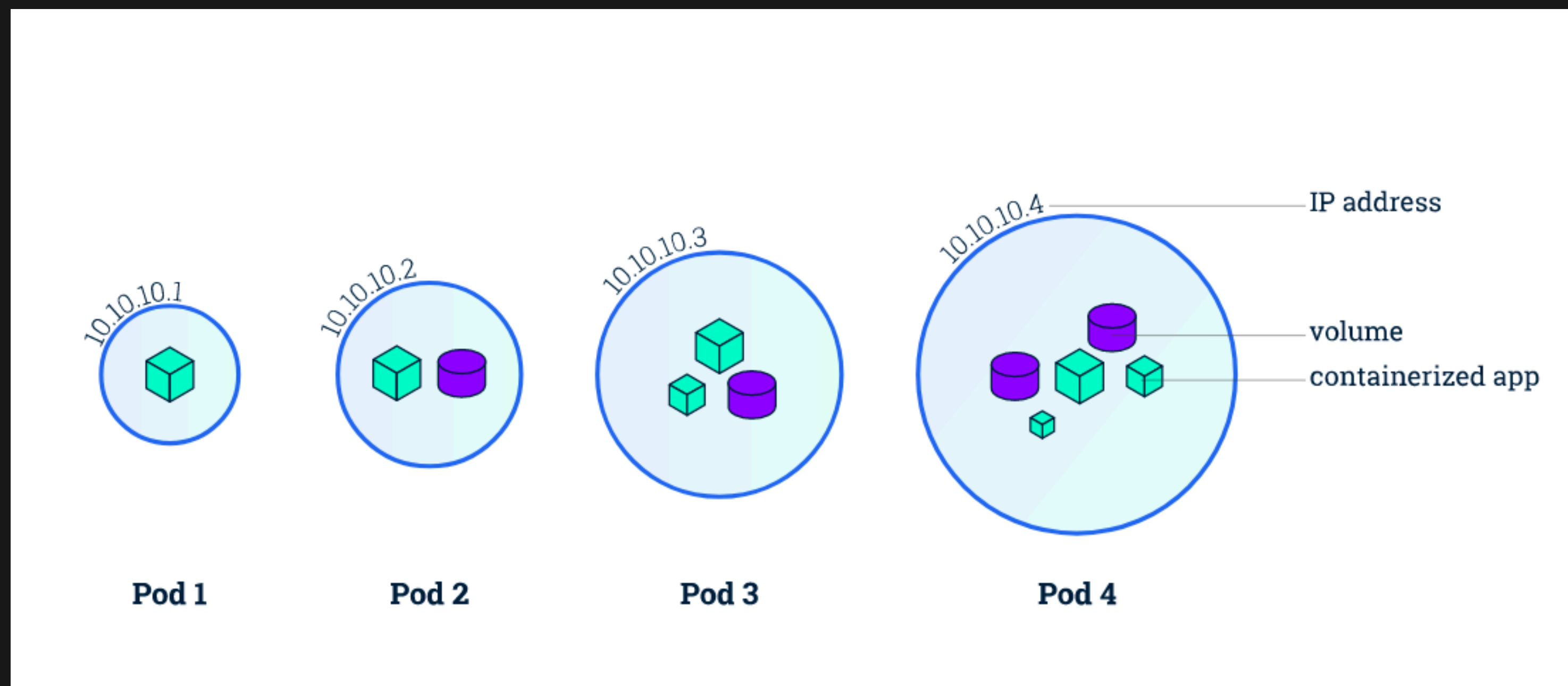
```

1  ► apiVersion: networking.istio.io/v1alpha3
2  kind: DestinationRule
3  metadata:
4    name: order20-stable
5    namespace: app-mutants
6  spec:
7    host: order20-stable.app-mutants.svc.cluster.local
8    trafficPolicy:
9      loadBalancer:
10       simple: ROUND_ROBIN
11     subsets:
12       - name: v1
13         labels:
14           app: order20

```

Virtual Service & Destination Rule

Istio Networking



Pod

Smallest deployable units of computing

Pod

- One or more containers
- Shared storage and network (ip)
- Co-located and co-scheduled
- etc.

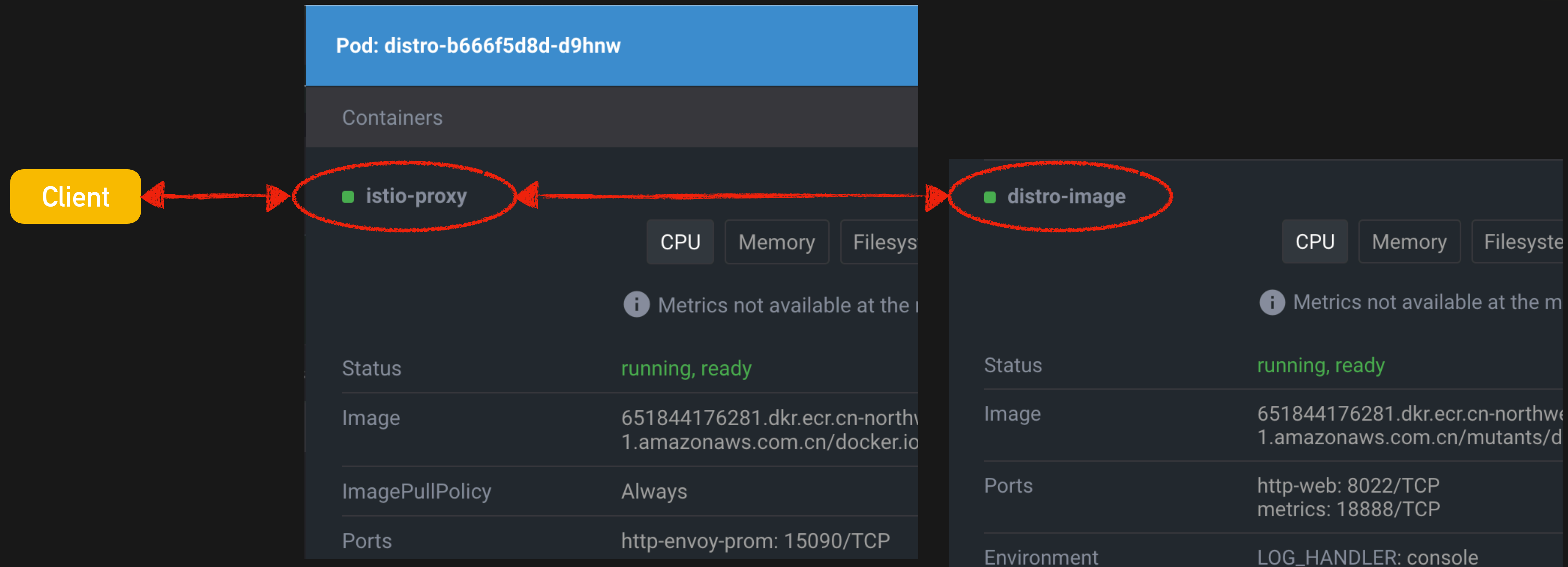
```

apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
spec:
  containers:
    - name: myapp-container
      image: busybox
      command: ['sh', '-c', 'echo Hello Kubernetes! && sleep 3600']

```

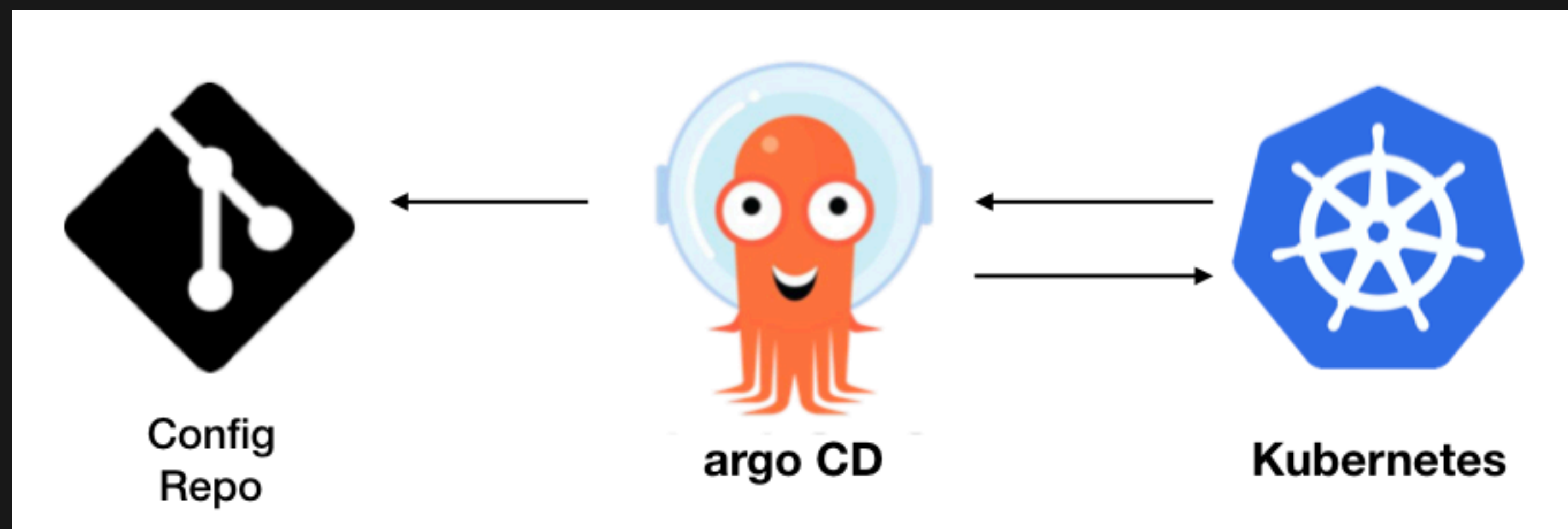
Pod

The very basic example



Pod

With Envoy as Sidecar



ArgoCD

Rollout

Rollout

- Deployment Strategy
 - Blue/Green
 - Canary
- Work with Service Mesh
- Examples

```

strategy:
  canary:
    stableService: order20-stable
    canaryService: order20-canary
    trafficRouting:
      istio:
        virtualService:
          name: order20-vsvc # required
          routes:
            - primary # At least one route is required
        steps:
          - setWeight: 80
          - pause: {duration: 3s}

```

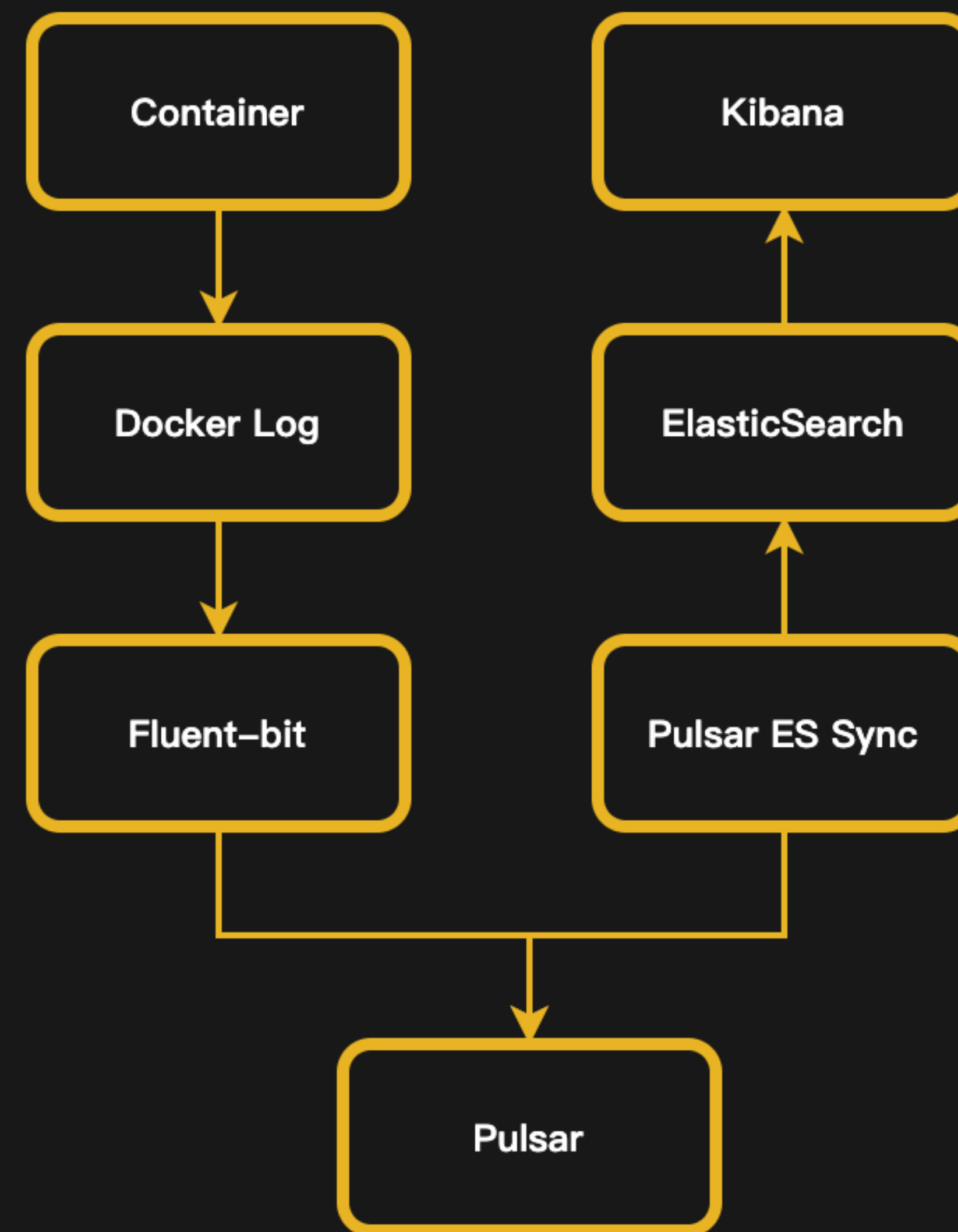
Yaml Examples

- Sandbox:
 - <https://gitlab.planetmeican.com/meican-cd/argocd-sandbox>
- Prod1:
 - <https://gitlab.planetmeican.com/meican-cd/argocd-production>
- Prod2:
 - <https://gitlab.planetmeican.com/meican-cd/argocd-prod>

Rich Features

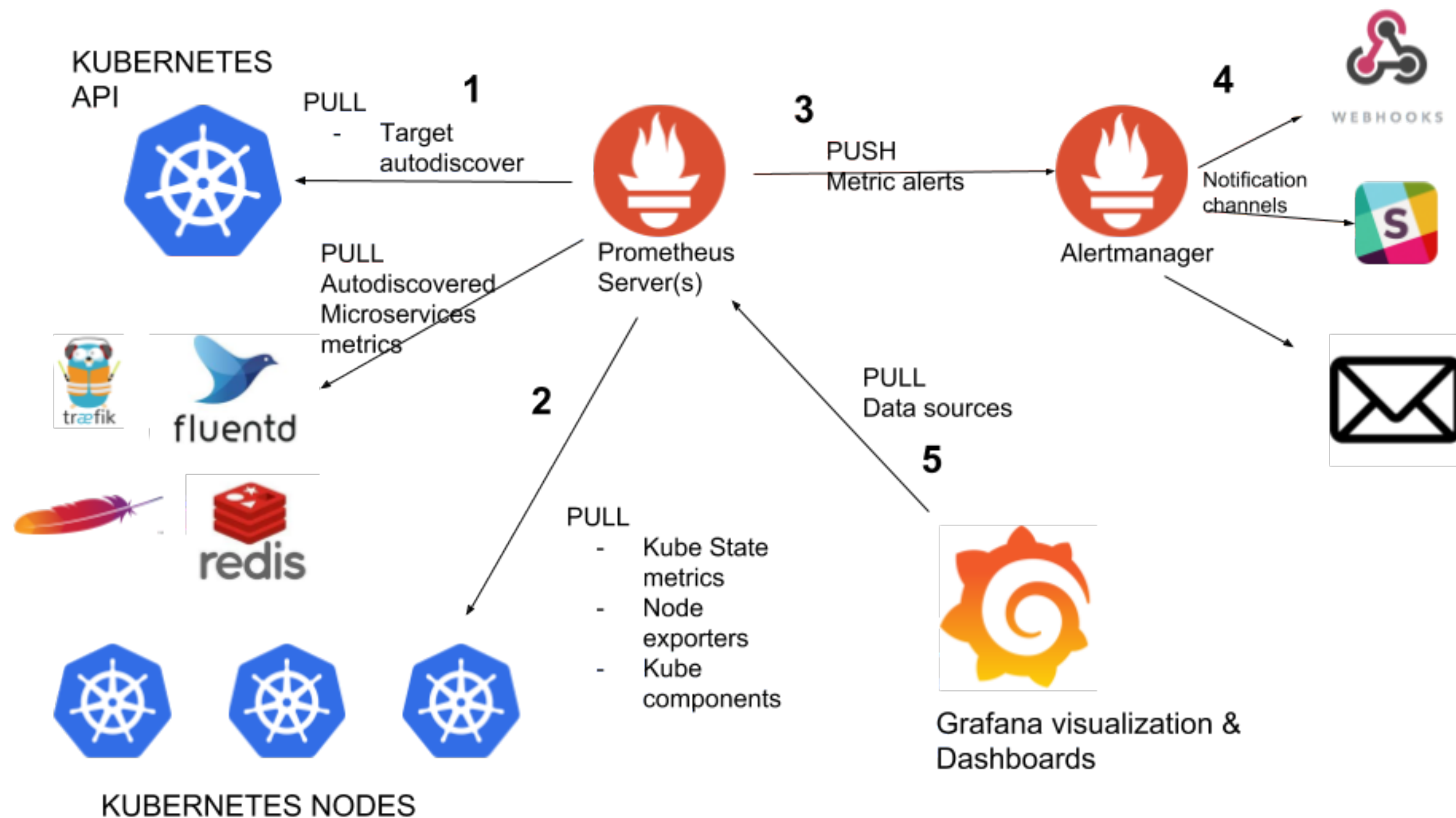
Logging with Fluent-bit

- Fluent-bit
- [Pulsar ElasticSearch Sync in Rust](https://gitlab.planetmeican.com/nerds/log)
- ElasticSearch
- Kibana
- <https://gitlab.planetmeican.com/nerds/log>



Logging Dashboards

- Sandbox:
 - <https://log.sandbox.planetmeican.com/>
- Prod1 (will be deprecated):
 - <https://log-nw1.planetmeican.com/>
- Prod2:
 - <https://log-nw.planetmeican.com/>



Monitoring

With Prometheus and Grafana and so on

Monitoring Dashboards

- Sandbox:
 - Prometheus: <https://prometheus.sandbox.planetmeican.com/>
 - Grafana: <https://grafana.sandbox.planetmeican.com/>
- Prod1 (will be deprecated):
 - Prometheus: <https://prometheus-nw1.planetmeican.com/>
 - Grafana: <https://grafana-nw1.planetmeican.com/>
- Prod2:
 - Prometheus: <https://prometheus-nw.planetmeican.com/>
 - Grafana: <https://grafana-nw.planetmeican.com/>

Monitoring an app

- Telemetry
- <https://gitlab.planetmeican.com/nerds/bazaar/-/blob/master/xtelemetry/telemetry.go#L234>

```

1  apiVersion: monitoring.coreos.com/v1
2  kind: ServiceMonitor
3  metadata:
4    labels:
5      name: order20
6      prometheus: k8s
7      name: order20-metrics-telemetry-monitor
8    namespace: monitoring
9  spec:
10   endpoints:
11     - port: telemetry
12     namespaceSelector:
13       matchNames:
14         - app-mutants
15     selector:
16       matchLabels:
17         app: order20

```

Monitoring an app

Old fashion way

```

daemonset.yaml x 26.InstallPulsar.md x configmap.yaml x statefulset.yaml x deployment.yaml x jaeger-query-kong2.yaml
1  apiVersion: apps/v1
2  kind: DaemonSet
3  metadata:
4    name: fluent-bit
5    namespace: logging
6    labels: 3 items
10 spec:
11   updateStrategy: DaemonSetUpdateStrategy
15   selector: LabelSelector
20   template:
21     metadata:
22       labels: 3 items
26     annotations:
27       prometheus.io/scrape: "true"
28       prometheus.io/port: "2020"
29       prometheus.io/path: /api/v1/metrics/prometheus
30     spec:
31       containers:
32       - name: fluent-bit
33         image: 321659100662.dkr.ecr.cn-northwest-1.amazonaws.com
34         imagePullPolicy: Always
35         ports:
36         - containerPort: 2020

```



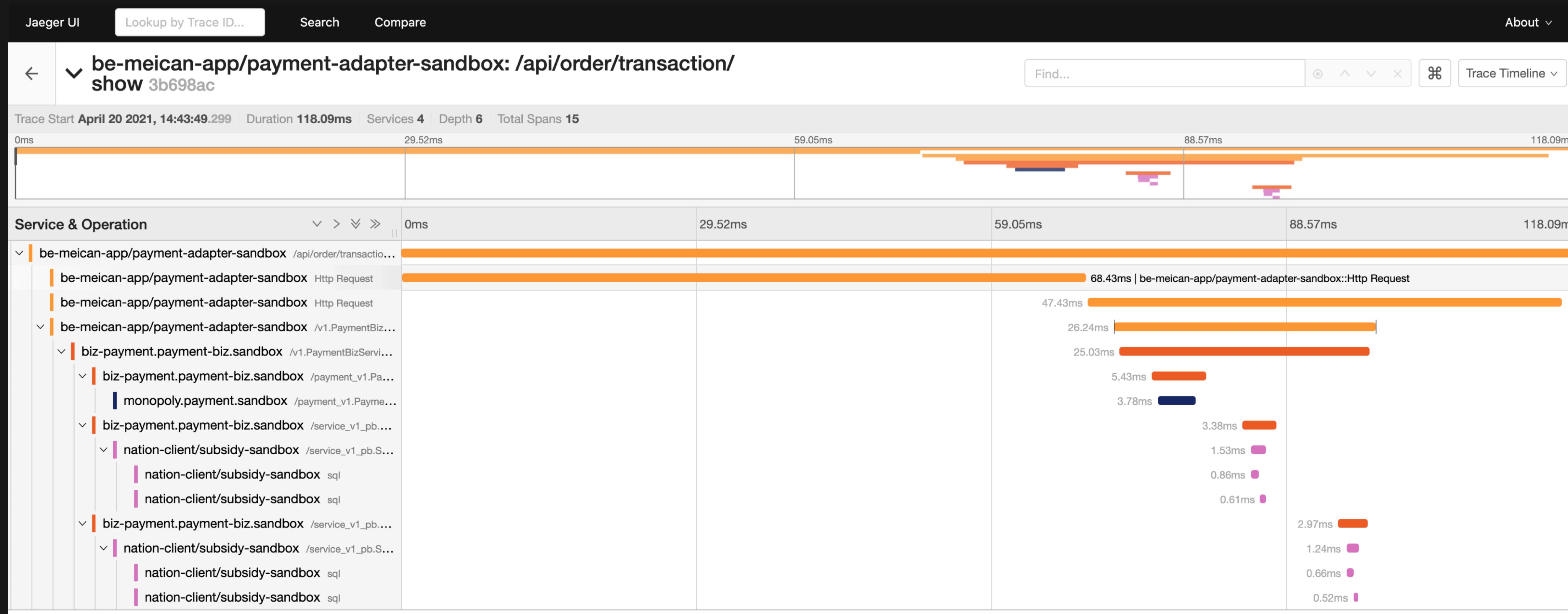
Jaeger

OpenTracing

Jaeger Tracing

- Connect to jaeger agent
- Automatically done by Tower

```
Environment    LOG_HANDLER: console
               LOG_FORMAT: json
               APP_GROUP: mutants
               APP_PROJECT: distro
               MY_POD_NAMESPACE: fieldRef(v1:metadata.namespace)
               JAEGER_AGENT_HOST: fieldRef(v1:status.hostIP)
               JAEGER_SERVICE_NAME: distro.app-mutants
               JAEGER_SAMPLER_MANAGER_HOST_PORT:
               http://$(JAEGER_AGENT_HOST):5778
               APP_RESOURCE_SCHEMA_PATH:
               /var/run/secrets/tower/tower_resource_schema.toml
               APP_CREDENTIALS:
               /var/run/secrets/tower/tower_resource_schema.toml
```

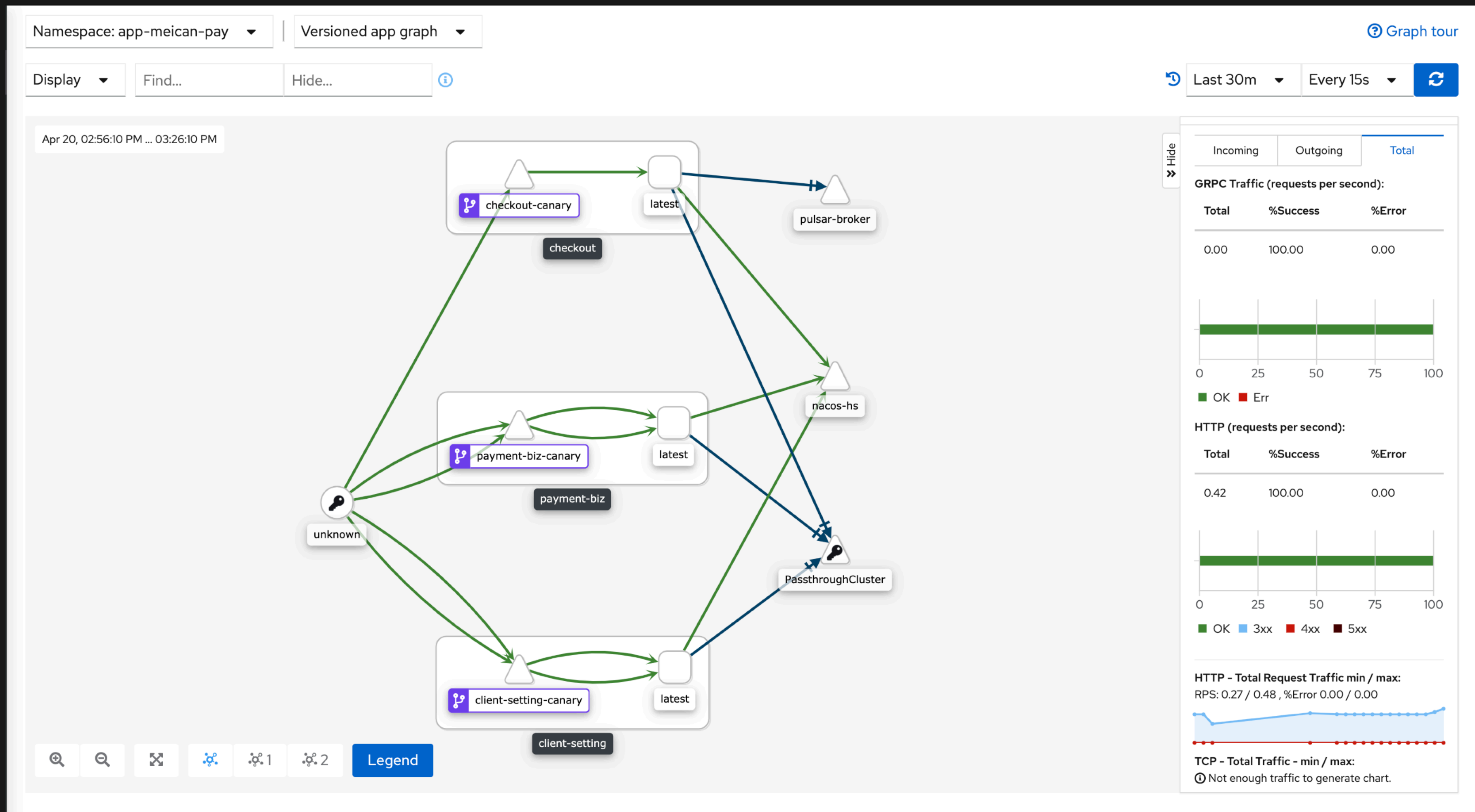


Jaeger Tracing

<https://gitlab.planetmeican.com/nerds/bazaar/-/blob/master/tracing/tracer.go>

Tracing Dashboards

- Sandbox:
 - <https://tracing.sandbox.planetmeican.com/>
- Prod1 (will be deprecated):
 - <https://tracing-nw1.sandbox.planetmeican.com/>
- Prod2:
 - <https://tracing-nw.planetmeican.com/>



Kiali

Service Mesh Observation

Kiali Dashboards

- Sandbox:
 - <https://kiali.sandbox.planetmeican.com/>
- Prod1 (will be deprecated):
 - <https://kiali-nw1.planetmeican.com/>
- Prod2:
 - <https://kiali.planetmeican.com/>



Nacos

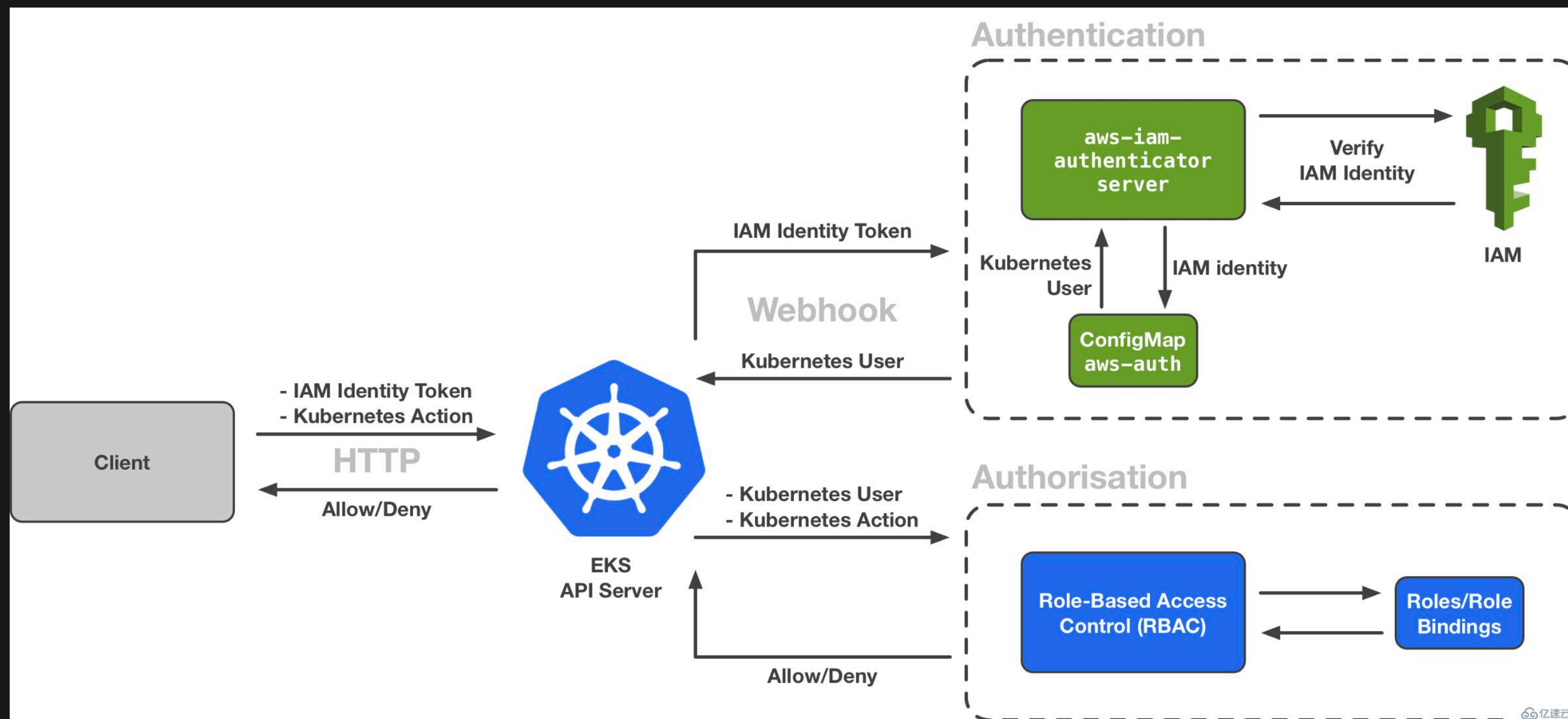
Configuration & Service Discovery (unimplemented yet)

Nacos

- SDK Client:
 - <https://github.com/nacos-group/nacos-sdk-go>
 replace =>
 - <https://gitlab.planetmeican.com/nerds/nacos-sdk-go>

Nacos Dashboards

- Sandbox:
 - <https://nacossandbox.planetmeican.com/>
- Prod1 (will be deprecated):
 - <https://nacos.planetmeican.com/>
- Prod2:
 - <https://nacos-nw.planetmeican.com/>



AWS IAM

AWS IAM

Via Service Account Annotation

Kind: ServiceAccount Name: fluentd Namespace: logging

```
1 kind: ServiceAccount
2 apiVersion: v1
3 metadata:
4   name: fluentd
5   namespace: logging
6   selfLink: /api/v1/namespaces/logging/serviceaccounts/fluentd
7   uid: 8cdec547-da73-4bcf-be9e-d70852212a6c
8   resourceVersion: '6070546'
9   creationTimestamp: '2020-04-22T08:13:02Z'
10 labels:
11   aws-usage: logging-fluentd
12   owner: eks-fan
13 annotations:
14   eks.amazonaws.com/role-arn: arn:aws-cn:iam::651844176281:role/eksctl-fan-addon-iamserviceaccount-logging-
15   arn:aws-cn:iam::651844176281:role/eksctl-fan-addon-iamserviceaccount-logging-
```

Environment

AWS_DEFAULT_REGION: cn-northwest-1
 AWS_REGION: cn-northwest-1
 AWS_ROLE_ARN: arn:aws-cn:iam::651844176281:role/eksctl-fan-addon-iamserviceaccount-logging-f-Role1-WUTQGTK2VVB0
 AWS_WEB_IDENTITY_TOKEN_FILE:
 /var/run/secrets/eks.amazonaws.com/serviceaccount/token

⚠ Injected by EKS Webhook

Messaging System

- Pulsar Unified endpoint:
 - `pulsar-broker.pulsar.ntrnl-eks-fan[clusterID].meican`

Tower Secret Management & Deployment Controller

```
webapp.yaml x
1  apiVersion: nerds.k8s.meican.com/v1alpha1
2  kind: WebApp
3  metadata:
4    name: helloworld
5    namespace: app-meican-cd
6  labels:
7    group: "meican-cd"
8    project: "helloworld"
9  annotations:
10    nerds.k8s.meican.com/group: "meican-cd"
11    nerds.k8s.meican.com/project: "helloworld"
12    nerds.k8s.meican.com/owned: "tower"
13  spec:
14    group: meican-cd
15    project: helloworld
16    mq:
17      namespace: meican-cd
18      tenant: public
19      topics:
20      - name: helloworld-t1
21        partitions: 1
22
```

Tower

- Work with annotations
- Inject proper environment variables and files
- Co-opt with nerds/app

Annotations

kubectrl.kubernetes.io/default-logs-container=order20-image

kubernetes.io/psp=eks.privileged

nerds.k8s.meican.com/group=mutants

nerds.k8s.meican.com/inject=true

nerds.k8s.meican.com/project=order20

prometheus.io/path=/stats/prometheus

prometheus.io/port=15020

prometheus.io/scrape=true

sidecar.istio.io/inject=true

sidecar.istio.io/logLevel=info

sidecar.istio.io/status={"initContainers":["istio-init"],"containers":["is...

Environment

APP_CREDENTIALS:
/var/run/secrets/tower/tower_resource_schema.toml

APP_GROUP: mutants

APP_PROJECT: order20

APP_RESOURCE_SCHEMA_PATH:
/var/run/secrets/tower/tower_resource_schema.toml

APP_SECRET: x08QI7hCr54L2z4

GRPC_GO_LOG_VERBOSITY_LEVEL: 99

GRPC_TRACE: all

GRPC_VERBOSITY: DEBUG

JAEGGER_AGENT_HOST: fieldRef(v1:status.hostIP)

JAEGGER_SAMPLER_MANAGER_HOST_PORT:
http://\$(JAEGGER_AGENT_HOST):5778

JAEGGER_SERVICE_NAME: order20.app-mutants

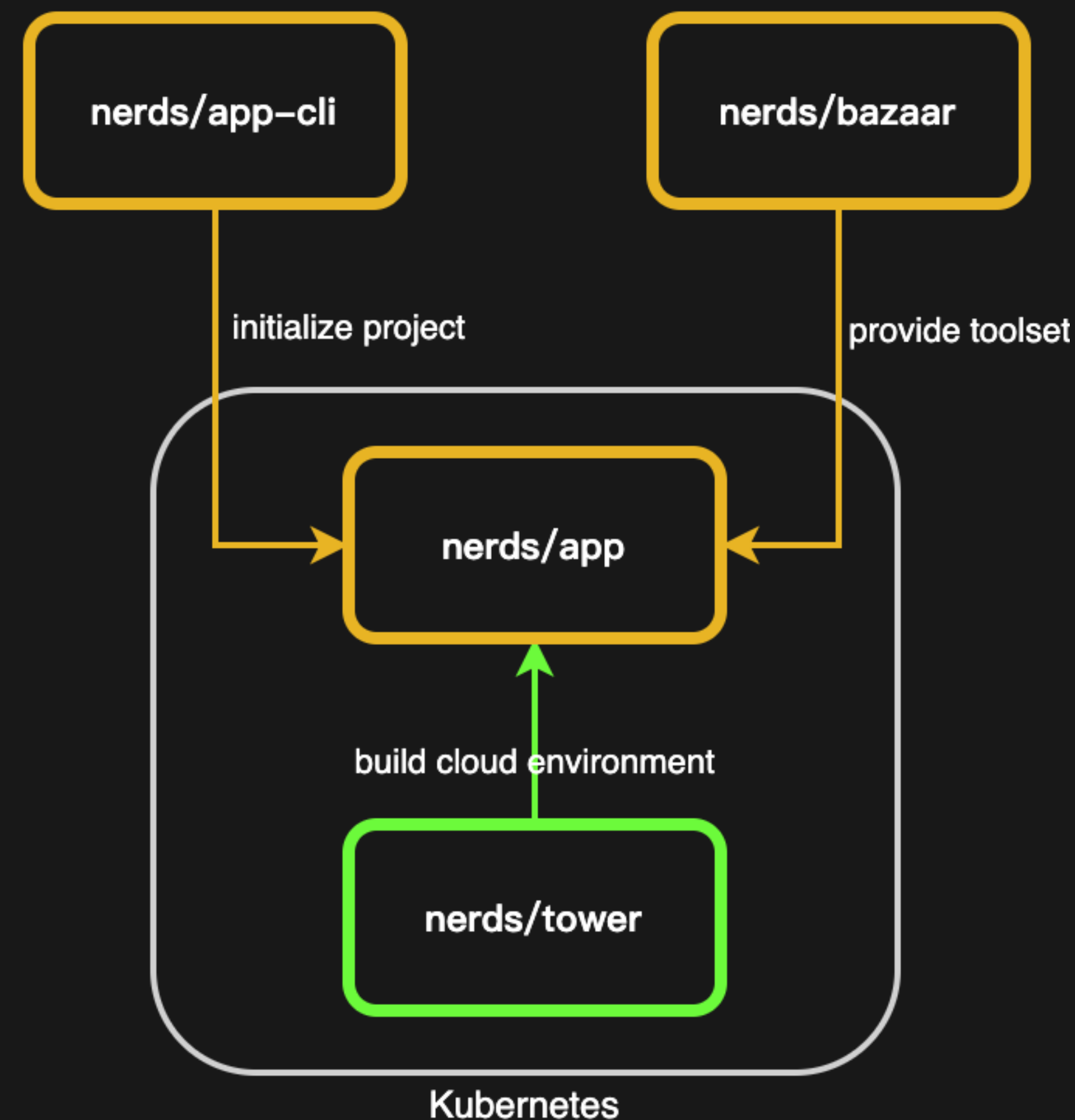
LOG_FORMAT: json

LOG_HANDLER: console

MY_POD_NAMESPACE: fieldRef(v1:metadata.namespace)

Scaffolding

- <https://gitlab.planetmeican.com/nerds/app-cli>
- <https://gitlab.planetmeican.com/nerds/app>
- <https://gitlab.planetmeican.com/nerds/bazaar>
- <https://gitlab.planetmeican.com/nerds/tower.v2>



Merge Request and Issue
Welcome !

Merge Request and Issue Welcome !



- Sandbox & Prod1: eks-fan
- Prod2: eks-fan2

Question ?

Fin.
Thank you.